

Video Transcripts

Module 15: Using Advanced Python Programming to Create Web Applications

Video 1: Introduction (02:11)

So, in this section, we're going to be developing two mini projects which you can add to your portfolio. So, in mini project one, we're going to be continuing our work with the Flask web server. We're going to be looking specifically at JSON web token authentication and authorization. So, instead of just telling the web server who we are, we're going to tell the web server also what we're allowed to do. We're going to use a simple application of publishing books, and we're going to have people that can read the books, but also, we're going to have people that can add to that list of books.

And so, they need to have admin authorization. So, we're going to have readers and admins as two kinds of user with different capabilities, different authorizations. We're going to be using these JSON web tokens to protect our APIs. We'll be going to be using decorators. We'll be using the JWT inbuilt decorators, but we'll also roll our own decorator. I really like decorators. Once you've mastered them, they're great. They really are powerful.

So, in this mini project, we're going to be using Python to develop a school grading system. It'll give us some experience with developing the kinds of data structures that we need. So, we'll be developing data structures based on dictionaries and lists and combinations of those. So, we'll have dictionaries within dictionaries. For example, we'll be using list and dictionary comprehension. These are one of the main styles of what we call Pythonic programming. So, once you've mastered these, you'll be an excellent Python programmer. Hope you enjoy this mini project. Okay, bye for now.

Video 2: JSON Web Tokens Architecture (04:12)

So, I want to take you through using JSON Web Tokens to authenticate and to authorize. Now, often there'll be two servers. So, this is the authorization server and then, we have the app here

that needs to understand if the user is authorized or not. So, the first stage is for the authorization server to receive a login request from the user. So, the user sends an HTTP message and in the body of that message, posts the username and password. The authorization server here then compares that with its list of authorized users and decides whether this person should be given a JSON web token or not. Now, a JSON web token is an encrypted set of data, and it's encrypted with a secret key.

Here, I've called it 'abc'. Now, that secret key is shared with the application server, and it could be many application servers. We might have a lot of web servers, but they would share this secret. Now if, for example, the username is 'John' and the right password is presented, then John will be granted a JSON web token. So, if they're granted a JSON web token, the username and the role that they're allowed will be encrypted with this secret and that is the JSON web token. And that's then sent back to the user. Now, the user needs to present that JSON web token in a request to the app server. The app server uses the shared secret key to decrypt the message and will then get the username and the role. And so, they know it's John, and they know John's role is administrator.

And let's suppose, that that is sufficient, then the app server will respond with the data that they request. Otherwise, the app server will respond with a negative message saying, data unavailable. Okay, so, that's the basic of how that works. Now, this is the encryption process. So, here John and Admin are encrypted with the secret key, some random string, and this is what the JSON web token looks like. It looks like gibberish, but it contains three parts, and we'll describe later what these three parts are. But this is what the user needs to send to the app server, and they need to put this in the header of the HTTP request.

So, this security token is embedded in the HTTP header. This is an example of doing that. We're going to do it in Postman. But if we wanted to write Python code, then this is how we would embed that security token, that JSON web token. We'd put it in the authorization part of the header, and it would be a bearer token, and then we'd send that HTTP request programmatically to the URL. Here I'm saying localhost:4000/books, but that's what it looks like in code. So, next we'll take a look at how this works in practice on an example problem. Bye for now.

Video 3: Project 1: Creating a Books Web Application (06:52)

In this mini project, we're going to go through a number of tasks related to the books. The first thing that we're going to do in task one is you're going to match up the images of the books so that they can be rendered like this. So, the page, when we go to say, books, the books route would print out the three books with the images like this. So, that's the first task. The second task is we're going to add some new users and we're going to give them roles. So, the roles are going to be either admin or reader. Now, an admin can add a book and add an image, but the reader cannot. So, the reader can look at the books with get books but can't add a book or add an image. So, these are the users, and you can put your own name instead of test user and give yourself a role of admin.

Here, I've got the role of reader and Anna's got the role of admin. So, now we need to arrange that only admin can add books or add images. How are we going to do that? Well, let's take a look at the code. So, in the code that I'm giving you, I've already extended it to include JSON Web Tokens. So, you'll see I'm loading a number of libraries here. You shouldn't need any more libraries than I've actually loaded. And we need to set up the secret key and we're going to store the JSON Web Tokens in cookies. And there are a few other configurations that we're setting. I've added some new books that I like, but you can add your own. There are three of them and I've given them an ID now.

That's because if we look inside static, I've got three images, and I want the image number to match up with the ID of the book. So, that we should be able to figure out which image belongs to which book when we're trying to render it. So, to render it, you're going to need to go into books.html. So, that's what's rendering these books. Here, I've put in the users.

You can alter these and now I've added into where we log in, we're going to set. It's not the session token, it's JWT. And in it we're going to store not only the user's name, but also, some claims. And the one of the claims that we're storing is the role of the user. So, I'm checking the user. So, when they log in, I check their username and password.

I create a valid user that consists of the username and their role. We don't want to store the passwords, so we've got the username and roles. I'm adding the role to the user claims in this JWT token, so that's done here. User claims, I'm sending it equal to and this is a dictionary. So, the name of the claim is role. So, when we're looking for it, that's what we're going to be looking for. And then, when we come to books, we have this decorator JWT required and that's part of the library. We won't be able to see that that function, but what I want you to do is to add now

this function as well, which checks if the JSON web token has the role of admin. So, that's what you need to do is you need to write this.

I'm going to give you some clues about how to start it. But what you'll notice here is that we can get the username out of the token, get JWT identity, and there's some other JWT functions that will allow you to pull back that role. You need to look them up and find them. So, your third goal is to, in addition to the JWT required which is given, I want you to write this decorator. Let's go back and see how we might do that. Here, is the outline of `admin_required`. This is how we're calling it. So, we need to complete that decorator. Here's some clues as to how you'd write that. You'd have `admin_required` and you're going to return the wrapper. If it's successful, you return this function.

Otherwise, in here we got to do some kind of test on the role of the user. And remember, we put that role in as a claim into the JSON Web token. So, you're going to need to learn about how this JSON web token library works. So, be careful to look at what library I'm using for the JSON Web Token. It's called `flask_jwt_extended`, `flask_jwt_extended`. That's the library. So, make sure you're looking up the right advice and then you should be able to complete this `admin_required`, this decorator. Okay, and what that will do then, is it'll protect the roots that have that decorator. The user will have to have admin permission to be able to hit those roots. Bye for now.

Video 4: Project 1: Creating a Books Web Application Solution (08:02)

So, our first task was that we needed to be able to display the books with the images. So, let's put in test user and we see that we get welcome test user. And if we hit the books, now we see that we get a nice print out; and here we've got the images being rendered and the details of the book. And we need to be able to then add a number of users. So, I added myself and I gave myself just reader permission. Now, you're going to be an admin, but a reader, I can't add a book, admins only. Similarly, I can't add an image, but I can still display the books. So, that works for John. Now, let's see an admin. I've got an admin called Anne and Welcome Anne. Add a book. Anne can add a book. Well, let's put something decent. Anne Harding, title, Lost Worlds.

So, we can submit that and we'll see that we get a book added. Let me scroll up a little bit just so you can see it. There you go. So, we have that and I've got some added images that I loaded in. So, it gets an image automatically there, but I can add an image that I like. It's the fourth book. So, I put 4 there and let me select an image. So, let's have a look here. Well, I've put something

that I know; I think this is me. So, we upload the image and then we can go back. So, I'll go back now and hit 'Books', and it looks like I overwrote the wrong one. We need to go and fix that. So, what I'm doing is I'm deleting the cache. So, I believe that it was rendering the wrong image because of caching. Let's see a missing cookie. I deleted my cookies, so I need to go back and log in. So, let's go back to 5000. Let's pretend that I'm Anne. Okay, submit. Let's take a look at the books. I put my picture there and now, it comes, it gets rendered.

So, be careful of caching that. You need sometimes to delete everything, otherwise you're getting a cached image of the page, but that's all working. So, that was all that we needed to do. We've got our roots protected. So, remember, I couldn't hit Add Book or Add Image, whereas Anne can and you should be able to as well because you're going to give yourself admin permission. Let's quickly look at the code that was necessary to do this. I'll put the solution up on GitHub so you can download it. But looking at the code here, you see, we had some images for the books and then we uploaded some other images. In books, the trick there was we needed to put an image tag and the source. Then we needed to make sure that we loaded the right image.

So, I decided to call the images by name, image, and I put the `{book [ 'id' ]}}`, so 0, 1 or 2; so that'd be image 1, etc, 2, and I put the width as 50. You can put it as whatever you like. So, that was to render the books. Then, we needed to go and add in users. So, that was fairly straightforward. We just put the users in. So, what we needed to do is we needed to make sure that the check on `validUsers`, so `checkUser`. Here, we needed to make sure that that worked. And then, I returned the user's name and their role, but not the password. We don't want to put the password in the token, that would be dangerous.

Then, what we do is, we put into the access token these added `user_claims` that I'm setting to this dictionary "role", and I'm putting into that the `validUser`'s role. So, that gets embedded into the JSON web token. We get the response object, so we had to add `make_response`. And then we're adding into the response the `status_code`, and also, we're getting the cookies. So, we set the access cookies in the response object and we put the access token there and then we 'return response'. So, that's going to get sent back to the user. The user's going to store it in their cookies and they're going to return it to us whenever they return. Now, what we needed to do was to protect the routes. So, here we had JSON Web Token required, `jwt_required`, but on `addbook` and `addimage`, we wanted to use this decorator `admin_required`. Let's go and look at that. You needed to understand the structure of the decorators. The decorator is itself using a decorator.

So, to use `@wraps` and this function, you had to load up. Here, you needed to import wraps from `functools` and then you needed to verify the JWT that it was in the request and then get out of that the claims.

And then, if the claims, and we know that we put role into the claims, if that was not equal to admin, then we return an error message, 'Admins only'. Otherwise, we return the function that's passed in with its arguments. So, that'll be whatever the function is below the decorator. So, that was it. So, download it from GitHub and run it for yourself. I'm sure there are other slightly different solutions, but as long as it works that would be fine. Okay, bye for now.

Video 5: Relational Data Structures (03:06)

So, here we're going to exercise both our Python writing skills and also our data structuring skills. So, this is typical data about a course that we're teaching, say, at MIT. So, we have the names of the students and we have the problem set number and the grade that the students get.

And then, of course, the Pset number needs to have another table that has, for example, the maximum score for that Pset. So, not every Pset has the same score. So, in this case, we'd have two tables, and the queries that we'd like to make from these tables are, for example, what's Fred's total for all the Psets. So, here we'd have Fred, and then we'd need to loop through and find Fred for Pset number 2 and Fred for Pset number 3, etc. So, we have to scan down this table to do that. Or what's Fred's average percentage for all the Psets? So, then we'd have to have Fred and scan down 1, 2, 3, etc, calculate the average, etc. So, there are questions that we'd like to ask of our data.

Now, an alternative to this would be, for example something like this, where we'd have the problem sets laid out in columns, so Fred and then Pset1, Pset2, Pset3, etc. And then, we'd still need this other related table with the PsetId and the MaxScore. Now, there's even another way we could lay this out, that we could have a table for students where the student has an ID. And in fact, that's what we do at MIT. We give every student an unique ID in case their name has a conflict.

And in my case, John Williams, there are three John Williams's in MIT that I know of, and there may be more. And so, we give an ID and typically we do this with all kinds of data. So, here we have the problem set, we have a problem PsetId with the grades. We might have a grade ID. Here we're not having a gradeId, but we're having a field. We're having 2 fields. A composite of these

two will be unique, the studentId and the PsetId will be unique. So, in this case, we're going to have to look up the grade by the studentId and the PsetId. Let's take a look at what kind of code we might write in Python.

Video 6: Creating a Database in Python (12:24)

So, let's start with a student. So, let's have a student table, and what's a student going to look like? Well, let's say student 1 will look like this. We'll have, for example, an ID and that'll be a number. So, this is the student number one and we'll have the student name and so the name would be, for example, Alice. So, that would be a good student and we'd have S2 and S2 would be. Let's go with the name and Bob would give us somebody starting with B and finally, we might have somebody, Student 3, name Charles. Okay, so we have three students and now we need to understand. Okay, are we going to have a student table of some kind? So, for example, we could have students would be maybe a data structure of some kind. What should we choose?

Well, we'd probably choose a dictionary, so we'd have students, and now we've got some choices about how to represent the students. But one choice would be to put them all in a list in an array S1, S2, S3. So, that would be a reasonable way of listing the students. What would we do now for the problem sets? Well, let's for the P sets we'd have problem set 1 equals and we'd follow a very similar style. We'd have an ID. Now, instead of name, we need the Max score, and the maxScore might be an integer, say 100. So, that'd be problem set one, problem set 2. We'll do three of each of these. Problem set two, problem set 3. So, here we go and just for fun, we'll have this one out of 80 and this one out of maybe 120 and now what would we have?

Well, now we'd have P sets, so this should be students P sets and we'd have the problem sets and again, we'd put them into an array, P1, P2, P3. That would be reasonable. Now, we've got grades. That's the next one we need to figure out. So, let's do that. We're going to have grades and what did we decide? Well, we're going to have the student ID. So, let's see how this grades is going to work. We'll have student ID, so say student 1 and now we'd need more, right?

We'd need also the problem set ID, so pid, so that might be one as well and finally we need the score. What did they get? And so, let's suppose they got 90 on that. So, that would be one grade, grade 2 and now we'd have our grades, for example, would look like we said we'd have grades and now we'd have an array, grade one, grade two, grade 3. Just make sure our structure is correct, right? That will work. An array, we can pick out grades.

Let's think out now about our database. Let's create a database. So, we'll have a database that looks like the following. Let's think about that. We want the tables in there. So, the tables that we want are. We want students, that's a table, that's an array rather and then we want to have psets. Let's make these strings. They're going to be keys, psets, and that would be another array of P1, P2, P3 and finally we'd have grades and that would be another array. So, that's what our database would look like. Let's print it out.

Okay, so let's have this as our database, but let's actually create it. Let's get ourselves, for example, a function that would create this database and then we can start filling in these values for the database. So, we'd have, for example, we define an empty database and we'd return and we'd return this structure. That would be a reasonable way to create our database. Now, we'd say `db = empty` and fire it and now we'd have our database, perfect.

Now, we need to add students to the database. How would we do that? Well, we need add student and we'd pass in the database and now we'd `addStudent`. So, what do we need for that? Well, we need to define a student. What else do we need to define? Well, we need the `studentId` and their name and now what would we do? Well, we'd add that to our database, so we do `db`. we need to insert it. So, we depend into the database and we append something of like the student that looked like this.

So, we're going to append. Something that looks like this. So, we put `studentId` and student name. Maybe we'll have we'll go for camel case and this would be student name. Okay, so that would add a student. Should we try it? So, add `Student` and now we need to pass the database. We've got the database, it's empty and we need to pass in the `studentId`. So, we need to pass in one and for example the name of the student which we had Alice. That would add Alice to the database. Let's just do a couple more Bob and Charles. Okay, let's print out the `db`. What do we need? How do we access that? We need this correct now. `db[ 'students' ]`. Perfect. So, we need to be aware of what exactly we're appending to.

So, that got us our students. We should probably check that these IDs are unique, but we'll do that later. So, you can see that we need to now do the same and let me, I'll do it fairly quickly. We need to add problem sets and add the grades.

Let's take a look at those. So, here I've added the pset. We passed the database, the problem set ID and the total points. So, now we append to pset ID and the points and here we do add `Pset` and that problem set one total points is 100. Problem set 290-3120. Let's do the same now for

adding the grade. So, to add the grade we'll need the DB again, the student ID, the problem set ID and the number of points, and we append to grades, and let's add some grades in.

Video 7: Querying a Database (04:40)

So, we're going to need to ask questions. So, the first thing that we need to do is to be able to get a grade. Let's just check what we get for database. So, this is our database now. And now, let's see if we can get the grade of a student. So, we're going to access this grades. So, 'def getGrade'. We need to pass in the database. We need to pass in the student ID and the problem ID. And now, we want to return what? Well, let's figure out what we're going to return. We need to access first from the database the grades. So, 'grades'. So, let's call this a row. So, from the row, let's suppose for 'row in db grades'. So, now we're picking out one of these. We're looping over the list and we're picking out a row. And from that row, what do we want?

Well, we want to return the points. So, we're going to need to get points and let's do a list comprehension. So, I need this is all going to be a list and we're going to have row points for row in database 'if'. Let's put that on the next line 'if'. Well, now we need row and we need the student ID. So, that's student is equal to 'sid' and row 'pset' is equal to pset ID, 'pid'. That looks good. Now we want to return this. Let's test it out. So, we're going to call getGrade and we'll pass in the database. And let's get student one, grade one. Let's just check it out. Now we got 98. And so, we're getting a row. Have we got two 98s? No. Yes, we have. Yes, we have. So, it got it right. We had a duplicate entry. Let's clean this up. And we need to re-zero our database. That's better, no getGrade. We have one grade 98.

So, be careful about re-running some of these cells. They'll just add duplicate data into the database. So, now we've got getGrade. I want you to change this. And instead of using a list comprehension, I want you to use a dictionary comprehension so that the result would look like this. So, I want this outputted instead of just the 98. I'd like this output as a dictionary. So, you need to change getGrade to have a dictionary comprehension. So, that's your first challenge.

Video 8: Creating a Student Grades Database Using Dictionaries (08:59)

So, let's take a look at creating another version of the database. In this case, instead of having the students as a list or an array, we're going to have a dictionary. And the same with Psets and grades. Now, remember with grades, we have the student ID and the Pset ID, and we could have the Pset first and the student ID second. But let's have the student ID first, and then we'll get the

Pset ID second. So, that's going to be our structure. So, let's take a look. We need to be able to add students, add Psets, and add grades just like before. So, we'll pass in the database the student ID and the student name. So, in this case, what do we need to do? We need to get hold of first `db[ 'students' ]`, since we're going to add it to the students. And now, it depends what we're going to have here. Let's have the student ID as the key. So, we'll go with `student_id` as the key. So, now we've got hold of the right dictionary, and now we just need the `student_name`. So, that should be it. Let's take a look at `addPset`.

So, with the pset, we're going to need to get hold of the database, but this time we'll get hold of the psets. And we need the `pset_id`. Okay. And that'll be `pset_total_points`. and that will be `pset_total_points`. And now, let's take a look at the `addGrade`. So, again we go to `db[ 'grades' ]`. Now we've got the right dictionary. Now, we said we'd get hold of first the `student_id`. So, now we've got the `student_id` and now there may not be a grade yet in there. So, let's think we need to check first if the `student_id` is in grades. So, if the `student_id` is not in the grades, we need to put the `student_id` there. Okay. And that will be for the `student_id`, then this is going to be empty.

Let's put make this right. We need a block for the if statement, that's correct. Now, if they are in the grades, we can go ahead and get the `student_id`. Okay, we've got that. Now as well, we need to get the `pset_id`. And finally, we can give them the points that they scored. and finally, we can give them the points that they scored. Lets check this. It should be fine. We're not executing anything here. Let's try to add a student. I've got some here, so let's add Alice to the database and see if we're okay. Let's list out the db. Now we need to be careful here. We really need to create the db first. So, let's create an empty db. So, and we need `db=empty()`. Now we've got a database. Let's rerun this. Since we can add... So, now we've added Alice, a student.

Let's try and add... Let's remove Alice from this. And I've already got these typed in, so let's see if we can add the pset grades and the grades here. So, now we've got students, psets, and grades. It looks okay. Let's see if we can now retrieve some of these. Let me give myself some clean space here. So, we've entered the students and the grades. We've got scores, now we need to get the grades. I'm going to write a helper function here, so this is going to help us when we're dealing with dictionaries. So, we're going to pass in a dictionary and a key, and we need to check if the key exists. So, if we do have a key, we want to return that dictionary with a key. Otherwise, we'll print out none. We'll return none. So, instead of throwing an error, this will save us quite a

bit. Let's take a look here. So, now let's get a student grade. Let's have a studentGrade. I want to get student grades. Let's see if I can get all the student grades. What do I need to do? Well, I'm going to return, but I need to get. And now, we're going to pass in the dictionary and the key is going to be the student_id. That should be it, because that will return to us the whole dictionary. Let's call db. Let's get student grades for one. We've got to get the grades first from the database, correct? Now we're in students; that's correct. We need to return this. Good, now we've got it. So, this is problem set one, problem set two and problem set three.

Excellent. So, now we've got the student grades. So, why don't you now write the following, studentGradeOn? So, we're going to have the studentid and the problem set ID. So, why don't you write that for for student 1 on problem set 3? Okay, check that out. Okay, bye for now.

Video 9: Redundant Dictionaries in Python Exercise (08:19)

So, let's see if we can make our dictionaries a little bit faster. Here we're going to use some redundant dictionaries. Now, we saw last time that we had dictionaries within dictionaries, that students itself is a dictionary. So, let's give ourselves a helper function here, where if we pass in the dictionary, we can pass two keys. For example, we might want to add a certain problem set to the problem sets. So, let's see how we do that. So, we're first going to check if key1 exists. So, if key1 not in the dictionary, then we put the key in there. We set up the dictionary.

Now, if it is in there, we go to it. And we add now to that dictionary a new key and we put it equal to the value that's being passed in. So, if the second key doesn't exist, it's going to create it and it's going to put the value in. So, this is the way that we initialize dictionaries.

So, that gives us a very nice helper function. Now, let's take a look at how we might add a student. We can borrow from the way we added student before, but let's just go through it. We're going to add to two dictionaries, studentById and studentByName. We're adding a student to two of those dictionaries. So, we're passing in a database. And we pass in the student_id and we pass in the student_name. Okay. Now what we want to do is we're going to go to the first dictionary, studentsById, and we're going to add the student_id and it's going to be equal to, this will be the student_name. And the next one we need db studentByName, and we're going to have student_name here, and that's going to be equal to student_id. So, now we've added a student. Let's now addPset. So, that's going to stay very much the same.

Okay. So, that's straightforward. Now, the last one, addGrade. Now, we're going to have gradesByStudent and gradesByPset. So, db student_id, pset_id and points. And now, we need to take advantage of our helper function. So, add2 db and now the first one is gradesByStudent. We're going to have student_id and points and this one is going to be add2 db again. But now, we're going to have gradesByPset and pset_id and we're missing one here. We need pset_id here. Because where this takes four arguments, key1 and key2. So key1, key2. And so, this one is just reversed. Now we've got pset_id, student_id points. Neat. Now, let's print the database. We haven't actually added the students, but let's see what we've got. We've got a typo. I've got a typo here somewhere there.

We're looking good. So, we've got an empty database. Now, we need to populate the database again. So, let's copy that from before. Up here we populated the database. Let's grab these again. Here's our redundant dictionaries. Let's go with all of these in here. I've got a typo here. I'm checking this. This is going to create the empty one. We should be fine. Let's take a look. Yes. Okay, now we've got studentsById, studentsByName, gradesByStudent, gradesByPset. Beautiful. Now we should be able to look them up much faster and much easier. Let's take a look at that. So, this is our database. So, let's write getGradesOnPset. So, we're going to pass in the database and we're going to pass in the pset_id.

Now we know we've already got here, we've got gradesByPset in the db database. So, we know that's what we want to use. But this will give us all the psets. Now, we want to pick out one key. So, remember we can use get on this. So, we pass in the dictionary and the key and the key is the pset_id. And of course we need to actually return it. So, now let's run this and you see we've got student one, student two, and student three. And if you check, that's exactly what we've got here, two and three. Okay, why don't you now write of one that will print out instead of printing out by student ID, print out the names. So, what I want you to write is getGradesOnPset but where we construct and have this with a name instead of the one, two and three. Bye for now.

Video 10: Redundant Dictionaries in Python Solution (04:47)

So, this is the exercise on student grades where we're using redundant dictionaries. And what we mean by that is we're storing the data in several different formats. So, for example, here we've got the studentsById; getting the student ID will give us the student name. This one studentsByName, given the student name, we can get their ID. Then with gradesByStudent, if we have the student ID we can get a list of their grades. And in this case given the pset_id, we can

get a list of the grades that the students got on that problem set. So, we did `getGradesOnPset` where we got the `pset_id` and this gave us a list of the students. So, student one got 125. This was on problem set three, student 2 got 115, student 3, 110.

Now, what we want is we want to add put the name of the student and the grade that they got to make this more intelligible. So, here we've got the function `gradesOnPset`, and now we've got to create this from this. Now, we know we've seen dictionary comprehensions where we take an input of a dictionary and we output a transformed dictionary. So, that's exactly what we need to do. So, let's first use this since that's where we want to start. And let's call this `students`. And now we've got this dictionary. What we want to do is to loop over that and produce another dictionary. So, we know it's a dictionary comprehension. And now, we're going to do a 'for' loop over this dictionary, so over students. And what we want in that is we want to get the key and we want to get the points. Now, the key is the student ID, so let's call it that. So, for student and points in students and we must get the items. We're good. Now, what do we want to output here? Well, we're going to output... Well, we need to transform the student ID to a name.

So, given the student ID, we can get the name by using the database and we'll get `studentsById`. So, now that's got us here, we've got this, and we just need to put in the student ID and that will return to us the name. So, we've got the `student_id`, we just got it here from `students`, and that's the name. And now, we just need to put the `:points` as the value. And we've got that as well. So, that should do it. Let me put this on the next line just so you can see. So, here we are, and we need to return that. So, that's our dictionary comprehension. Let's see if we're right. Yep, executed it and here we are. So, it's fairly straightforward if you understand what you're trying to achieve. So, given this, we could transform it into this by using the `studentsById` dictionary. You can imagine there's lots of other transformations that we'll need, and we may do an exercise on that, but this is the solution to this present exercise. Okay. Hope you enjoyed it. Bye for now.